

Day 5 — Networking, SCADA, Safety & Capstone: Full Instructor Teaching Material (Technical Edition)

Total session time: 210 minutes (3.5 hrs). Each section is sized to its time budget with talking points, board work, protocol/register detail, safety-wiring detail, and explicit framing lines.

1. PLC Networking — 15 min

1a. How PLCs Talk To Each Other And To SCADA/HMI — 10 min

Everything built Days 1–4 has lived on one isolated PLC. State the shift explicitly before going further: a real facility rarely has just one PLC — it typically has several, each controlling one machine or line, plus **HMIs** (Human-Machine Interfaces — operator touchscreens local to a machine) and a **SCADA** system (Supervisory Control and Data Acquisition — plant-wide monitoring and control software, usually running on a server in a control room).

The core mechanism, stated precisely: a PLC exposes its internal tag data over a network connection; HMI and SCADA clients connect to the PLC (directly, or through an OPC server acting as a middle layer — name only today) and either **poll** for tag values on a fixed interval or **subscribe** so the PLC/gateway pushes changes as they occur. State explicitly: the PLC's control program itself does not change to support this — the same tags being read/written by ladder logic are simply also exposed for external read/write access, which is exactly what makes tag binding (Block 4) a configuration-level task rather than new PLC programming.

1b. Real-World Network Topology Example — 5 min

Diagram to draw on the board:

```
[PLC - Line 1] --Ethernet--\  
[PLC - Line 2] --Ethernet---+--[Managed Switch]--[SCADA Server, control room]  
[PLC - Line 3] --Ethernet--/ |  
| [Historian / logging]  
[HMI touchscreen, local to each machine]
```

Frame this explicitly as the "big picture" context that makes the rest of today's SCADA content make sense: students have worked with exactly one isolated PLC all week until now, with no networking layer at all above it.

2. Communication Protocols — 15 min

2a. Modbus — 5 min

One of the oldest and simplest industrial protocols, still extremely widely used because it is simple and vendor-agnostic. Register/data model, worth putting on the board precisely:

Data type	Access	Size	Typical use
Coil	Read/Write	1 bit	Digital outputs
Discrete Input	Read-only	1 bit	Digital inputs
Holding Register	Read/Write	16-bit word	Analog outputs, setpoints, general data
Input Register	Read-only	16-bit word	Analog inputs

Common function codes, worth naming a few explicitly since they appear directly in diagnostic tools: **01** Read Coils, **02** Read Discrete Inputs, **03** Read Holding Registers, **04** Read Input Registers, **05** / **06** Write Single Coil/Register, **15** / **16** Write Multiple Coils/Registers.

Two transport variants — **Modbus RTU** (serial, RS-232/RS-485, includes a CRC checksum per message) and **Modbus TCP** (Ethernet-encapsulated, standard port 502, relies on TCP's own error-checking rather than an application-level CRC). State explicitly: the underlying register model (coils, holding registers, etc.) is identical between the two — only the physical transport and framing differ.

2b. Profinet — 5 min

Siemens-originated, Ethernet-based, designed for real-time industrial communication with tighter timing guarantees than plain Modbus TCP. Name two real-time classes at a recognition level only: standard **RT** (Real-Time) and **IRT** (Isochronous Real-Time, for applications needing microsecond-level synchronization such as motion control). Devices are described by a **GSDML** file (a standardized XML device-description format) that the engineering software imports so the network configuration tool knows a given device's capabilities without manual entry. Common in TIA Portal/Siemens-heavy environments.

2c. EtherNet/IP — 5 min

Rockwell/Allen-Bradley-originated, also Ethernet-based, built on the same underlying **CIP** (Common Industrial Protocol) shared across several other industrial networks (DeviceNet, ControlNet). Two messaging types worth naming: **explicit messaging** (request/response, used for configuration and diagnostics, not time-critical) and **implicit/I/O messaging** (cyclic, time-critical data exchange — the actual real-time control data).

Framing for students, stated explicitly: none of these three protocols need configuration-depth mastery today — the goal is recognition. If any of these terms come up on the job, students should immediately know what category of thing is being discussed (an industrial network protocol) and roughly what role it plays, not necessarily how to configure it from scratch.

3. Safety Basics — E-Stop & Safety Relay Wiring — 25 min

3a. Dual-Channel E-Stop Wiring — 10 min

Day 1, Block 4d established why an E-stop is wired NC — a wiring fault fails safe because it reads identically to a real actuation. Today extends that principle one level further: a **single-channel** E-stop (one NC contact, as wired Day 1) is a reasonable default for hobby/training work, but real industrial safety circuits typically use **dual-channel** wiring — two independent NC contacts on the same physical E-stop button, each feeding a separate monitoring channel.

Why two channels, stated precisely — the failure mode a single channel cannot catch: a single-channel circuit that reads "stop requested" correctly (logic 0) can still have a *different*, independent internal fault — e.g., a channel that has itself failed shorted, permanently appearing "not pressed" regardless of the button's actual state. With only one channel, there is nothing to compare that reading against, so this specific failure mode goes undetected. With two independent channels wired to the same physical button, a correctly-functioning safety relay compares both channels every cycle: if they ever disagree (one reads "pressed," the other reads "not pressed") when they should always agree, that mismatch itself is treated as a fault condition and the safety relay drops output power — a single failed channel does not silently disable the safety function, it triggers a detectable fault instead.

3b. Safety Relay Basics — 8 min

A **safety relay module** is a dedicated hardware device, physically separate from the PLC's general-purpose I/O, that monitors the dual-channel E-stop circuit described above using its own certified, hardwired logic, and directly cuts power to the machine's motor contactors when a stop condition (or a channel-mismatch fault) is detected.

State the key architectural point clearly, since it is easy to assume otherwise after four days of writing everything in PLC logic: safety-critical stop functions are generally wired through this dedicated hardware, not solely through the PLC's program logic — because a software bug, a PLC fault, or a PLC that is simply in the wrong scan phase at the wrong moment should never be able to defeat an emergency stop. The PLC can *read* the safety relay's status bit (to know the machine is currently stopped, for HMI display or program logic purposes) but it is not the thing standing between "E-stop pressed" and "power actually cut" — the safety relay is.

Name, at recognition level only, the formal safety-rating vocabulary students will encounter on real specification sheets: **Category** ratings (e.g., Category 3, Category 4 per ISO 13849) and **Performance Level** (PL d, PL e) describe how robust a given safety circuit design is against single-fault conditions — full depth is genuinely out of scope this week, worth a one-line forward-pointer alongside OPC UA and Structured Text in today's closing wrap-up rather than taught now.

3c. Project: Dual-Channel E-Stop — 7 min

Task: wire a dual-channel E-stop into the existing motor circuit from Day 1's seal-in project, feeding two separate input channels rather than one. Verify the fault-detection behavior directly — disconnect (or force false) only one of the two channels while leaving the other reading normal, and confirm the safety logic reports a fault condition rather than either (a) silently continuing to run, or (b) simply reading as a normal stop indistinguishable from a real E-stop press. Ask students to state explicitly, in their own words, why this specific test is what proves the second channel is doing real work rather than being redundant wiring with no functional purpose.

4. SCADA Integration — 40 min

4a. HMI/SCADA Tag Binding Concept — 10 min

Tag binding, defined precisely: an HMI screen element (a lamp graphic, a numeric readout, a button) is linked directly to a specific PLC tag by reference. When that tag's value changes in the PLC, the bound HMI element updates automatically on the next poll or push cycle — no separate communication logic needs to be written in the PLC program for it; this is a configuration-level link made in the HMI/SCADA authoring software, not new

PLC programming, directly following from the "PLC exposes tags for external access" mechanism established in Block 1a.

Concrete example to put on the board: a red lamp graphic on an HMI screen bound to `HighTempAlarm` — the PLC program never contains any instruction that "draws a red lamp"; the binding itself is entirely the HMI tool's responsibility, reading a BOOL tag it did not create and rendering it according to a rule an operator/engineer configured once.

4b. Alarm Handling & Event Logging — 10 min

Distinguish two related but different concepts precisely, since SCADA systems treat them differently:

- **Alarm** — an abnormal condition requiring operator awareness and typically requiring acknowledgment (an explicit "I have seen this" action from an operator before it clears from an active-alarm list), e.g. `HighTempAlarm` going true.
- **Event** — a purely informational log entry with no acknowledgment requirement, recording that something happened at a point in time (a batch completed, a mode switch occurred, an operator logged in).

Directly reuse Day 3's alarm-logging pattern (Block 5, the `R_TRIG` -gated log entry on first transition to true) as the mechanism that actually produces a loggable event from a continuously-true BOOL condition — state this callback explicitly: "this is the exact same one-shot logging pattern from Day 3's temperature project, now formalized as a real SCADA alarm/event record rather than a single internal tag."

A typical SCADA alarm record, worth putting on the board as a concrete row format:

```
Timestamp | Tag | State | Acknowledged
2026-08-14 09:14:22 | HighTempAlarm | ACTIVE | No
2026-08-14 09:16:05 | HighTempAlarm | CLEARED | Yes
```

Note explicitly: a complete alarm record needs both an "became active" entry and a "cleared" entry to be genuinely useful for later review — directly reusing the `F_TRIG`-for-"alarm cleared" extension question that was raised but left optional at the end of Day 3's project (Block 5, Test & Debug). Today is where that optional extension becomes a formally expected part of a real alarm/event system.

4c. Project: Bind Tags To An HMI Screen — 20 min

Task: bind three tags from Day 4's Batch Mixing Station FBS (and Day 3's temperature monitor) to an HMI screen:

- A status lamp bound to `Mixer.MixMotor` (or an equivalent "mixing in progress" bit) — lamp shows lit while mixing is active.
- A numeric readout bound to a level or `ScaledTemp` tag — displays the live engineering-unit value, directly reusing Day 3's REAL scaling output.
- An alarm indicator bound to `HighTempAlarm` — lamp or banner entry showing active/cleared state per Block 4b's record format.

Steps to guide students through: (1) create a tag database entry in the HMI software mapped to the actual PLC symbol/address for each of the three tags; (2) drag the corresponding graphic element (lamp, numeric display, alarm banner) onto the screen; (3) bind each element's relevant property (fill color, displayed value, banner text) to its tag. While circulating, verify students are binding to the *correct* underlying tag rather than a similarly-named one — a mismatched binding is a genuinely common real-world commissioning mistake that

produces a screen that looks correct at rest (e.g., a lamp defaulting to "off") but never updates, which is easy to miss without actively forcing the underlying tag and confirming the screen responds.

Why this project matters as a moment, worth calling out to students explicitly: this is the first time all week that PLC-internal data is rendered somewhere outside the PLC programming software itself — genuinely often a real "aha" moment connecting the whole week's work (scan cycle, ladder logic, scaling, FBs) into something that visibly resembles an actual industrial operator screen.

5. Capstone Project — Automated Sorting/Packaging Line — 90 min

The goal: teams combine essentially everything from the week — motor control and seal-in (Day 1), timers/counters/edge detection (Day 2), one analog input with scaling and comparison (Day 3), function blocks (Day 4), and today's dual-channel E-stop and SCADA tag binding — into one working system, with minimal step-by-step guidance from the instructor.

1. Team Planning — 10 min

Assign roles (wiring, logic, HMI) and have teams sketch the overall system on paper before building anything. Encourage teams to explicitly list which Day 1–4 concept each part of the planned system will reuse — a good forcing function for teams to recognize concretely how much they have actually accumulated over the week, rather than treating the capstone as an entirely new task.

2. Build Motor Control + Sensors + Timers/Counters — 25 min

The core sorting mechanism: a conveyor motor (seal-in circuit per Day 1), a part-present sensor gated through edge detection into a **CTU** (per Day 2), and a diverter/reject actuator timed by a **TON** (per Day 2). Encourage teams to package this as an FB per Day 4 if time allows, rather than flat rungs — not required, but worth suggesting as the more scalable approach given today's content.

3. Add Analog Input + E-Stop Safety Circuit — 25 min

An analog sensor feeding a sort decision — e.g., sorting by a simulated size or weight reading, scaled per Day 3's **SCL** pattern and compared against a threshold to decide which of two output lanes a part is diverted toward. Wire the dual-channel E-stop from Block 3 into the whole system so it can genuinely stop the conveyor motor regardless of what state the sorting logic is in — this is the practical test of whether the safety circuit was actually integrated correctly rather than left as an isolated demo from earlier in the day.

4. Add SCADA Monitoring, Integrate & Test — 30 min

Bind key tags (conveyor running status, part count, current alarm state, E-stop status) to an HMI view per Block 4's pattern, then run the full line end-to-end from a part entering to it being sorted into the correct lane, with the HMI screen genuinely reflecting live system state throughout the run.

Your role during this block, stated explicitly: deliberately step back from leading — circulate, answer direct questions, but let teams work through integration issues themselves. This is the one project of the week explicitly designed to be less guided than every previous project, since the goal here is testing whether the week's individual pieces have actually connected into working understanding, not whether a memorized sequence of steps was followed correctly.

If a team finishes early: add a reject/second-sort branch — sort into 3 categories instead of 2 based on the analog reading, using two comparison thresholds instead of one. This extends the exact same pattern already

built without requiring any new instruction the team hasn't already used this week — a good use of remaining time before demos rather than idle waiting.

6. Team Demos + Wrap-Up — 25 min

6a. Live Team Demos — 15 min

Each team runs their line live for the group. Keep it brief per team — the value here is seeing different valid approaches to the same underlying problem (there is no single correct sorting-line architecture), not an exhaustive rung-by-rung review of every team's program.

6b. Q&A + Where To Go Next — 10 min

Close with a short, honest note on what this week deliberately did not cover in hands-on depth, and where to go deeper from here:

- **Structured Text (ST) as a primary language** — used throughout this week alongside ladder for showing underlying logic precisely, but never taught as a full standalone programming language in its own right; worth exploring further for anyone who found the ST snippets easier to read than the ladder diagrams.
- **Full TIA Portal / CODESYS feature depth** — this week used a generic subset of whichever platform the lab ran; both tools have substantially more configuration and diagnostic tooling than a first week can cover.
- **OPC UA** — a modern, more secure industrial communication standard increasingly replacing older protocols like Modbus in new installations, mentioned here by name only.
- **PID tuning and formal safety Category/Performance Level design** — both flagged as concept-only this week (Day 4 Block 3, Day 5 Block 3b) with their full depth pointed to explicitly as a next step rather than taught.

Frame this closing list explicitly as "here's your map for continuing," not a discouraging list of gaps — every one of these builds directly on a foundation this week actually did cover in depth.

Note On Scope

Safety PLC / dedicated safety-controller design (as distinct from the hardwired safety relay covered in Block 3) was intentionally left out of hands-on teaching this week due to time, mentioned only as a "where to go next" pointer alongside OPC UA — the hardwired safety-relay approach taught in Block 3 was judged the more immediately useful foundation for students heading toward general shop-floor and integration work, with software-based safety controllers as a natural next step once that foundation is solid.